Comparison of Supervised ML Classifiers using the Iris dataset

Python codes for comparison of Supervised ML Classifiers using the Iris dataset. Plotting the Confusion Matrix and calculating the performance metric parameters (Jupyter Notebook)

1. Import Libraries

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score

from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
```

2. Load the Iris Dataset

In [2]:
```
iris = load_iris()
X = iris.data
y = iris.target

class_names = iris.target_names
```

3. Split the Dataset (Train – Test)

In [3]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

4. Define All Classifiers

In [4]:
```
models = {
    "kNN": KNeighborsClassifier(n_neighbors=3),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "SVM": SVC(kernel='rbf', random_state=42),
    "Naive Bayes": GaussianNB()
}
```

5. Train Models

In [5]:
```
for name, model in models.items():
    model.fit(X_train, y_train)
```

6. Confusion Matrix Plot Function

In [6]:
```
def plot_conf_matrix(ax, y_true, y_pred, title):
    cm = confusion_matrix(y_true, y_pred)

    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        xticklabels=class_names,
        yticklabels=class_names,
        ax=ax
    )

    ax.set_title(title)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
```

7. Plot Confusion Matrices for All Models

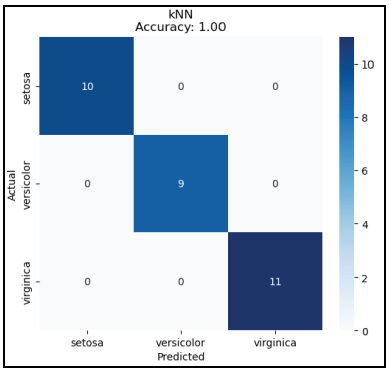
In [7]:
```
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
axes = axes.ravel()

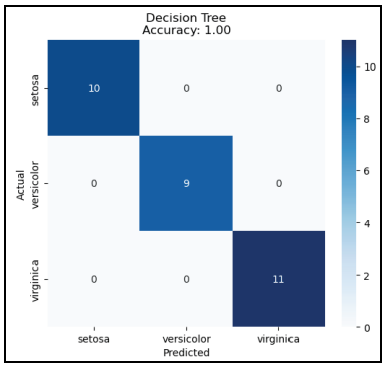
for i, (name, model) in enumerate(models.items()):
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

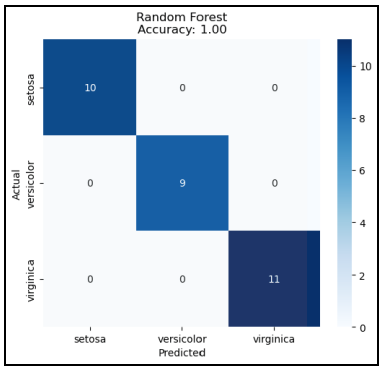
    plot_conf_matrix(
        axes[i],
        y_test,
        y_pred,
        f"{name}\nAccuracy: {acc:.2f}"
    )

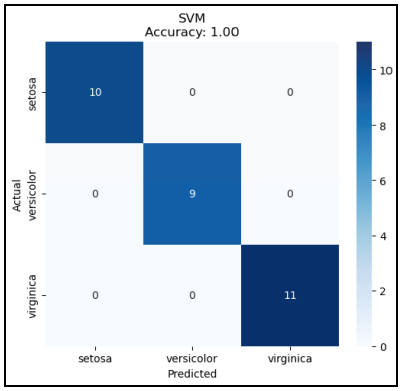
# Remove empty subplot
fig.delaxes(axes[-1])

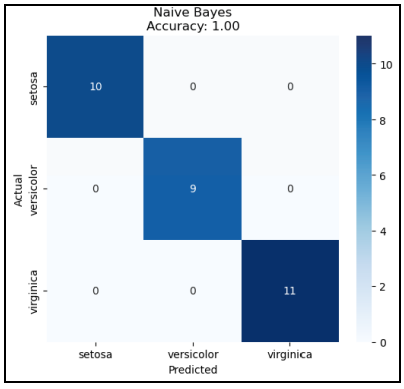
plt.tight_layout()
plt.show()
```











8. Compute Performance Metrics from Confusion Matrix

In [8]:
```python
from sklearn.metrics import precision_score, recall_score, f1_score

def compute_metrics(y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred)

    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred, average='macro')
    recall = recall_score(y_true, y_pred, average='macro')
    f1 = f1_score(y_true, y_pred, average='macro')

    return cm, accuracy, precision, recall, f1
```

9. Evaluate All Models

In [9]:
```python
results = {}

for name, model in models.items():
    y_pred = model.predict(X_test)

    cm, acc, prec, rec, f1 = compute_metrics(y_test, y_pred)

    results[name] = {
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1-score": f1
    }
```

10. Display Results

In [10]:
```python
for model_name, metrics in results.items():
    print(f"\n🔷 {model_name}")
    for metric_name, value in metrics.items():
        print(f"{metric_name}: {value:.4f}")
```

```
🔷 kNN
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-score: 1.0000

🔷 Decision Tree
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-score: 1.0000

🔷 Random Forest
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-score: 1.0000

🔷 SVM
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-score: 1.0000

🔷 Naive Bayes
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-score: 1.0000
```

11. Convert to DataFrame (clean table)

In [11]:
```python
import pandas as pd

df_results = pd.DataFrame(results).T
print(df_results)
```

	Accuracy	Precision	Recall	F1-score
kNN	1.0	1.0	1.0	1.0
Decision Tree	1.0	1.0	1.0	1.0
Random Forest	1.0	1.0	1.0	1.0
SVM	1.0	1.0	1.0	1.0
Naive Bayes	1.0	1.0	1.0	1.0